A Deep Learning-Based Educational Cryptocurrency Chatbot

Sunday 18th October, 2026 - 12:36

Vladyslav Siulhin
University of Luxembourg
Email: vladyslav.siulhin.001@student.uni.lu

This report has been produced under the supervision of:
Salima LAMSIYAH
University of Luxembourg
Email: salima.lamsiyah@uni.lu

Abstract

This project has been completed by student Vladyslav Siulhin, with guidance from his project academic tutor Salima Lamsiyah, and it presents final version of report for the Bachelor Semester Project 3 (BSP3).

The primary objective of this project is to create an educational cryptocurrency chatbot. The focus is on providing accurate and informative responses to queries typically posed by learners seeking basic information about cryptocurrency. The purpose of the scientific work is to answer to the scientific question. The scientific inquiry will delve into areas such as Cryptocurrency, Artificial Intelligence (AI), Large Language Models (LLM), Retrieval-augmented generation (RAG) and Natural Language Processing (NLP), aiming to gain insights that can inform the development of an effective educational chatbot. The aim of the technical work is to develop a chatbot that answers to the questions related to cryptocurrency.

1. Introduction

Nowadays, chatbots are getting more popular. One of the example that proves it is ChatGPT. To keep up with the times the decision was made to work on something similar to the new trend, but more narrowly focused. For example, in the modern world, a lot of people are interested in exploring information about cryptocurrency. This digital phenomena has become a powerful force in the modern world, not only providing a means of earning money but also reshaping the economic landscape of certain counties through its adoption as a revolutionary payment method.

To create educational cryptocurrency chatbot it needs to divide work on the project for 2 parts: scientific and technical.

The scientific part is aimed to study about Ctyptocurrency, AI, Deep Learning (DL), NLP, LLM and RAG.

Regarding technical part it needs to create educational cryptocurrency chatbot, based on the knowledge that was gained from the previous part, that is able to answer to the simple questions, normally produced by people that are not familiar with cryptocurrency.

In this way, at the end of the project it is possible to explore the expanding field of chatbot technology as well as enhance understanding of cryptocurrency, AI, LLM, RAG and NLP. At the same time meeting educational needs of users.

2. Project description

2.1. Domains

2.1.1. Scientific . The scientific domains of this project are Cryptocurrency, AI, LLM, RAG, NLP.

- **Cryptocurrency** is a decentralized digital currency crafted for online transactions.
- **Artificial intelligence** is a broad area of computer science focused on creating intelligent machines that can do things usually done by humans using their intelligence.
- **Large Language Model** is a type of artificial intelligence model designed to understand, generate, and work with human language. These models are ”large” both in terms of the size of the dataset they are trained on and the number of parameters they contain.
- **Retrieval-augmented generation** is a method that enhances the knowledge of Large Language Models by incorporating extra data, which is frequently private or current.
- **Natural language processing** is a specialized field within AI, computer science, and linguistics. Its primary goal is to enable computers to understand and interpret human communication, including spoken and written language. For this project only written language will be used.

2.1.2. Technical. The technical domains include Data Management and Modelling, Implementation of RAG technique, User Interaction and Interface.

- **Data Management and Modeling** involves tasks related to searching for a good dataset, preprocessing the data and fine-tuning the pretrained Neural Network model.

- **Implementation of RAG technique** includes creating a chatbot for the particular usage, namely, cryptocurrency that is related to this project.
- **User Interaction and Interface** focuses on designing an intuitive and user-friendly interface for effective interactions with the chatbot.

2.2. Targeted Deliverables

2.2.1. Scientific deliverable. The scientific part of this project aimed at exploring the capabilities of deep learning in the development of a chatbot focused on cryptocurrency education. This research endeavors to understand how advanced machine learning techniques, particularly deep neural networks can be utilized to create a chatbot that answers questions about cryptocurrency and assists students in their learning process.

Project's scientific objective includes Large Language Models, Retrieval-augmented generation and Natural Language Understanding which is investigating the natural language processing capabilities of deep learning models to enable the chatbot to understand and respond to user requests effectively.

2.2.2. Technical deliverable. The technical work in this project encompasses several critical components:

1) **Data Preparation:** This involves searching for a suitable dataset or creating one and preprocessing the data to ensure its quality and relevance to the project.
2) **Chatbot Architecture:** Designing the underlying architecture of the educational chatbot which includes components for natural language understanding, for example.
3) **Deep Learning Models:** Implementing and fine-tuning deep neural networks, including Pre-trained Large Language Models like Llama2, for instance.
4) **Retrieval-augmented generation:** connecting the dataset with LLM to create a chatbot that answers to questions about cryptocurrency.
5) **User Interaction:** Creating an interface that allows users to interact with the chatbot via text commands.

3. Pre-requisites

3.1. Scientific pre-requisites

To answer to the scientific question it is essential to possess knowledge of the following topics: big range of knowledge about cryptocurrency, understanding of basic concepts of Artificial Intelligence and Deep Learning; deep knowledge of main work principles of Neural Networks; understanding of how Natural Language Processing can be used for designing educational chatbot, understanding of how LLM and RAG can be used to create a chatbot.

Before starting answering to the scientific question author knew some basic information about AI and Neural Networks. The remaining knowledge was acquired during the project's implementation phase, specifically, learning information about cryptocurrency, understanding of how NLP allows the chatbot to comprehend and interpret human language, understanding of LLM and RAG. Those steps are crucial for processing user queries and generating meaningful responses.

3.2. Technical pre-requisites

To create an educational chatbot, it is crucial to have a strong knowledge in the following topics: good Python skills, ability to work with Python libraries, a good grasp of mathematics, acquaintance with Neural Networks, understanding of NLP algorithm that allows to chatbot to process user queries and generate meaningful responses, ability to work with LLM, knowledge about how RAG can be applied.

Initially, before doing this project, the author of the report had knowledge in the following areas: Python, mathematics and ability to work with Python libraries. Yet, in order to accomplish the project effectively, the author had to gain additional knowledge about NLP algorithms and certain Python libraries that allow to work with it. Additionally LLM and RAG techniques were learnt.

4. A Scientific Deliverable 1

This section is the main part of the scientific deliverable of the report, it consists of all the significant information about the scientific part of the project. All main findings in the project are fully explained. They are made clearer with simple descriptions and pictures. This approach offers a comprehensive view of the outcomes and their relevance.

4.1. Requirements

In order to answer to the scientific question which is: **"How can Natural Language Processing and Deep Learning techniques be used to create an educational cryptocurrency chatbot?"** this document has to incorporate both textual content and visual elements. It is expected to fulfill the following requirements: *Functional requirements* and *Non-functional requirements*.

4.1.1. Functional requirements.

1) **Learn how to develop Deep Learning-Based Educational Cryptocurrency Chatbot**
   To develop such chatbot it requires integrating a deep understanding of NLP, cryptocurrency concepts, and advanced technologies like LLM and RAG. This unified approach is designed to develop a chatbot capable of effectively comprehending and responding in human language, specifically tailored to the cryptocurrency domain. The chatbot will leverage NLP as its foundational technology for meaningful interactions, incorporate detailed knowledge about cryptocurrency to provide accurate information, and utilize LLM and RAG to enhance its conversational abilities and relevance in the digital currency space.

2) **Literature review**.
To understand NLP, LLM, RAG and discover how to make the chatbot understand human language and give useful answers, it's important to locate comprehensive information sources. These can include scientific books and detailed articles that explain the topic thoroughly.

4.1.2. Non-functional requirements.

1) **Scientific Validity and Clarity**
To maintain the scientific integrity of the project and ensure the clarity of research findings, it is important to follow to established standards of scientific rigor and clarity.
To do this, it needs to read and understand books and scientific articles focused on NLP, chatbot development, LLM and RAG accurately citing all relevant sources. It also involves using of appropriate research methodologies and data analysis techniques.
The use of clear and concise language, along with visual aids like pictures and diagrams, can improve the overall clarity of the project, making them accessible to the intended audience.
2) **Accessibility and User-Friendliness**
To guarantee that the project outcomes are accessible and user-friendly, straightforward language should be used. The content has to be easily understood by individuals with varying levels of familiarity with the topics. The user interface of the educational chatbot should prioritize simplicity and intuitiveness to ensure users of all backgrounds can engage with it comfortably.

4.2. Design

To fulfill scientific requirements the following scientific books and articles were used. Each unit of source of information presents specific topic linked to scientific question. The production part was designed using references in the following order:

1. Introduction to NLP

One of the main topic related to this project is Natural Language Processing. To get started, the book that calls "Handbook of Natural Language Processing" [1] that was written by *Nitin Indurkhya, Fred J. Damerau, Thore Graepel, Ralf Herbrich* was used. This book describes NLP in details and gives general idea of it. Also "Hand-book of Natural Language Processing" provides useful methods and instruments for integrating natural language processing into computer systems.

2. Introduction to Cryptocurrencies and Blockchain Technology

To understand the concept of cryptocurrencies and the underlying technology, the book titled "The Basics of Bitcoins and Blockchains: An Introduction to Cryptocurrencies and the Technology that Powers Them" [2] was referenced. Authored by *Antony Lewis*, this book provides:

- A foundational understanding of digital currencies, especially Bitcoin and Ethereum.
- Insights into the revolutionary blockchain technology that powers these digital currencies.
- A historical perspective on the evolution of cryptocurrencies.
- Practical aspects of cryptocurrencies, including how to buy, sell, and mine them.

3. Introduction to concept of chatbots

Rashid Khan and *Anik Dasbook* are authors of the book "Build Better Chatbots: A Complete Guide to Getting Started with Chatbots" [3] they wrote it in 2018. The book provides a comprehensive introduction to chatbots and guides through the process of creating chatbots. It covers topics such as the history and recent developments of chatbots, setting up a developer environment, and the basics of bot building, including intents and entities. The book also delves into advanced bot building techniques, design principles, and the integration of features like location lookup and intent classification. Additionally, it discusses the business and monetization aspects of chatbots, including analytics and use cases in various industries.

4. Large Language Models

LLMs, such as GPT-4 or Llama2, are pivotal in today's chatbot technology, empowering them to process and generate human-like language. Referencing "What are Large Language Models?" [4] from Elastic, this project explores how LLMs understand complex queries and generate informative, context-relevant responses. LLMs' ability to parse and produce language that closely resembles human conversation makes them invaluable for creating chatbots that can effectively communicate and educate users on intricate topics like cryptocurrencies.

5. Retrieval-augmented generation

To ensure the chatbot provides the most current and accurate information, especially in the rapidly changing domain of cryptocurrency, RAG should be studied. As explained in the LangChain documentation "Retrieval-Augmented Generation for Question Answering" [5], RAG combines real-time data retrieval with the generative power of LLMs. This approach enables the chatbot to pull the latest information from various sources, like financial databases and online news, and seamlessly integrate this with its responses. RAG's inclusion ensures the chatbot is not just knowledgeable, but also up-to-date with the latest market trends, technological advancements, and regulatory changes in the cryptocurrency world.

6. Building an Educational Cryptocurrency Chatbot

Nowadays, chatbots have emerged as powerful tools for enhancing user interactions and providing instant responses. The fusion of NLP and DL has been instrumental in elevating the capabilities of these chatbots, enabling them to understand and process human language with unparalleled precision. This combination of technologies is particularly useful in the field of cryptocurrency, a domain that demands clarity, accuracy, and relevance of information.

In conclusion, creating an educational chatbot focused on cryptocurrency using NLP, DL, LLM and RAG is complex but this experience is valuable. Such a chatbot showcases how technology can connect people with the complex world of cryptocurrencies. It makes sure that information is not only

accessible but also accurate, relevant, and engaging.

4.3. Production

4.3.1. Introduction to NLP. Natural Language Processing is a pivotal domain within artificial intelligence that focuses on enabling machines to understand, interpret, and generate human language. Its primary goal is to bridge the gap between human communication and computer understanding. This interdisciplinary field combines elements of computer science, linguistics, and data science to create systems that not only comprehend written and spoken language but also respond and interact in a way that mimics human conversation.

At its core, NLP involves the development of algorithms and models that can analyze, decipher, and make sense of human language in a meaningful way. This process includes a range of tasks such as speech recognition, language translation, sentiment analysis, and text summarization. By parsing text and speech, NLP systems can extract meaning, identify patterns, and even understand the emotions and intentions behind the words.

4.3.2. Introduction to Cryptocurrencies and Blockchain Technology. Cryptocurrencies are a groundbreaking form of digital or virtual currencies. They leverage cryptography to ensure security, making them resistant to counterfeiting. These digital assets are decentralized by nature, operating independently of a central authority or financial institution.

At the heart of most cryptocurrencies lies the **Blockchain Technology**. A blockchain is a distributed ledger or database that is open to anyone. It records transactions in a series of blocks, with each block containing a cryptographic hash of the previous one. This structure ensures an interconnected and tamper-proof chain.

Key Features:

1) **Decentralization**
   Traditional systems often have a central authority. In contrast, blockchain operates across a distributed network of nodes, ensuring no single point of control or failure.
2) **Transparency**
   Every transaction on the blockchain is public. However, the identities involved in these transactions are encrypted, ensuring privacy.
3) **Security**
   Before a transaction is added to the blockchain, it must be verified by a consensus mechanism. Once approved, it's encrypted and linked to the preceding transaction, ensuring a secure and immutable record.

Practical Aspects of Cryptocurrencies

Engaging with cryptocurrencies involves various practical aspects, from buying and selling on exchanges to mining, which is the process of validating and recording transactions on the blockchain. The security of one's assets is paramount, emphasizing the role of digital wallets and the safeguarding of private keys.

While cryptocurrencies are a notable application of blockchain, the technology's potential extends far beyond. Blockchain can power smart contracts, decentralized applications (DApps), and offer transformative solutions for industries like supply chain, healthcare, and real estate, capitalizing on its features of transparency and security.

4.3.3. Introduction to concept of chatbots. Chatbots are artificial intelligence systems designed to simulate human conversation. They can interact with users, answer queries, and perform tasks through textual or auditory methods. With advancements in AI and NLP, chatbots have become increasingly sophisticated, capable of understanding context and intent.

Building Chatbots

1) **Platform Selection**
   There are various platforms available for building chatbots, from cloud-based solutions like Dialogflow to open-source frameworks like Rasa.
2) **Training**
   Chatbots require training data to understand user queries better. This involves feeding them with sample conversations to recognize patterns.
3) **Integration**
   Once built, chatbots can be integrated into various platforms, including websites, messaging apps, and even voice assistants.

Advanced Chatbot Features

1) **Contextual Understanding**
   Advanced chatbots can maintain context in conversations, allowing for more natural interactions.
2) **Multimodal Interaction**
   Some chatbots can interact through both text and voice, providing users with multiple avenues of communication.
3) **Analytics and Feedback Loop**
   Monitoring chatbot interactions can provide insights into user behavior and preferences, allowing for continuous improvement.

4.3.4. Large Language Models. Large Language Models (LLM) such as LLaMA 2 or GPT-4 represent significant improvements in AI and NLP, offering advanced capabilities in language understanding and generation. At their core, LLMs are trained on vast amounts of textual data, enabling them to learn the distinctions, styles, and complexities of human language. This training involves processing large amount of text, covering a diverse range of topics, languages, and styles, which equips them with a broad understanding of language and context. LLMs typically use advanced neural network architectures, such as transformers, which allow them to process and generate language in a way that considers the broader context of the text. This ability to contextualize significantly enhances their language comprehension and response generation, making them highly effective for tasks requiring deep linguistic understanding. While LLMs are pre-trained on

general data, they can be further fine-tuned on specific datasets or domains. This fine-tuning process customizes the model to better understand and respond to language concerning to particular topics, industries, or user needs, thereby enhancing its effectiveness in specialized applications. For this project two LLMs were chosen, namely Llama 2 and GPT-4.

- **LLaMA 2 Overview**
  LLaMA 2 is an innovative language model that excels in understanding and generating complex language constructs, making it ideal for applications like chatbots. Its architecture allows for a deep understanding of context and nuance in human language, simplifying more natural and accurate interactions.
- **GPT-4 Overview**
  Alongside LLaMA 2, GPT-4 is also a significant achievement in the field of LLMs. As the latest iteration of the Generative Pre-trained Transformer series by OpenAI, GPT-4 is known for its extensive training on diverse datasets. It excels in generating human-like text, making it highly effective for applications requiring complex language processing and generation.
- **Application in Chatbot Technology**
  Within the chatbot development, LLaMA 2's advanced capabilities enable the creation of highly interactive and intelligent chatbots. These chatbots can involve in meaningful conversations, understand complex user queries, and provide detailed, contextually relevant responses, particularly useful in explaining complex cryptocurrency concepts.

4.3.5. Retrieval-augmented generation. Retrieval-Augmented Generation is a technique used in AI to augment the knowledge of Large Language Models with additional data, which can be private or real-time. It's particularly useful for building AI applications that need to reason about private data or data introduced after the model's training cutoff date. RAG involves bringing appropriate information and inserting it into the model prompt. This process enhances LLMs by enabling them to access and integrate external information, making them more versatile and effective in responding to specific or updated queries.

RAG, in combination with GPT-4, allows the chatbot to retrieve relevant information from the dataset in response to user queries. This integration enhances GPT-4's generative capabilities, ensuring that the chatbot's answers are informed by the most relevant and comprehensive information available in the dataset.

Principle of work:

1) **Load**
   Initially, diverse data formats are loaded into the system.
2) **Split**
   The loaded data is then segmented into manageable chunks, making it easier to process.
3) **Embed**
   Each chunk is transformed into embeddings, which are numerical representations that capture the semantic meaning of the text.
4) **Store**
   These embeddings are stored in a vector space, creating a vector store that the chatbot can query to find the most relevant information for any given user input.

The principle of RAG is to utilize these embeddings during the chatbot's operation. When a query is received, the chatbot retrieves the closest matching embeddings from the vector store, ensuring the generated response is informed by the most relevant and accurate data.

4.3.6. Building an educational cryptocurrency chatbot. The project demonstrates the development of two educational cryptocurrency chatbots using state-of-the-art AI technologies, namely, fine-tuning LLM Llama 2 and using of RAG technique on the GPT-4. Each chatbot, based on different LLM technologies and methodologies, using the same comprehensive dataset, highlighting varied approaches to AI-based educational tools in the field of cryptocurrency.

The integration of Deep Learning, particularly through neural networks, augments the chatbots' capabilities to predict and interpret user inquiries. The use of LLaMA 2 and GPT-4 demonstrates the versatility and power of modern LLMs in creating nuanced and interactive chatbot experiences.

Chatbot Design Principles

1) **User-Centric**
   The chatbot was designed with the user in mind, ensuring that it provides answers in a format that's easy for the user to understand.
2) **Informative**
   Given the complex nature of cryptocurrencies, the chatbot is equipped with up-to-date and accurate information.
3) **Adaptive**
   As the world of cryptocurrency is ever-evolving, the chatbot should have mechanisms to update its knowledge base regularly.

4.4. Assessment

The document was produced responding to the scientific question and successfully met all the defined requirements.

Functional Requirements

1) **Developing Deep Learning-Based Educational Cryptocurrency Chatbot**
   The document provides a comprehensive understanding of NLP, highlighting its significance in enabling chatbots to have meaningful interactions with users.
   It also delves into the world of cryptocurrency, explaining its decentralized nature, historical evolution, and potential future.
   This document contains exploration of Large Language Models and Retrieval-Augmented Generation as well. It explains how LLMs, with their advanced language

processing capabilities, are crucial for developing a chatbot that can understand and generate nuanced human language. Furthermore, the document effectively explains the concept of RAG, demonstrating how this technique improves chatbots by adding their responses with information retrieved from extensive datasets.

2) **Literature Review**
A thorough literature review has been conducted, drawing from reputable scientific books and articles. This has provided a solid foundation for understanding NLP, deep learning, and their application in chatbot development.

Non-functional Requirements

1) **Scientific Validity and Clarity**
   a) The document upholds high standards of scientific rigor, ensuring the validity and integrity of the research.
   b) All sources of information, including books and scientific articles, have been accurately cited, reflecting a commitment to academic honesty.
   c) The research methodologies and data analysis techniques employed are appropriate and align with established scientific standards.
   d) The inclusion of visual aids, such as diagrams and images, enhances the clarity and comprehensibility of the research findings.
2) **Accessibility and User-Friendliness**
   a) The language used in the document is clear and concise, making it accessible to a broad audience, irrespective of their familiarity with AI and NLP.
   b) The educational chatbot's user interface prioritizes simplicity and intuitiveness, ensuring that users from diverse backgrounds can interact with it seamlessly.

In conclusion, the document effectively addresses the scientific question, providing a full understanding of how to develop an educational cryptocurrency chatbot. All functional and non-functional requirements have been met, underscoring the document's comprehensive nature and its adherence to scientific standards.

5. A Technical Deliverable 1

This section is the main part of the technical deliverable of the report, it consists of all the significant information about the technical part of the project. The development of programs are fully explained. They are made clearer with description of the code.

5.1. Requirements

The program is supposed to be implemented with programming language Python. It also should have Graphical User Interface (GUI). The main purpose of the program is to provide user with information about cryptocurrency in form of chatbot.

5.1.1. Functional requirements.

1) **Providing user with meaningful responses**
The chatbot should be capable of delivering accurate and informative responses to user queries related to cryptocurrency.
It should be reached utilizing DL, NLP, LLM and RAG techniques to understand and interpret user input effectively.
2) **Creating Graphical User Interface**
The program should have a user-friendly GUI that enables users to interact with the chatbot easily.
It should have an intuitive design and layout, ensuring a nice user experience while accessing cryptocurrency information.

5.1.2. Non-functional requirements.

1) **Responsiveness**
The chatbot should respond promptly to user queries, ensuring minimal lag in providing answers.
It also should process user inputs and generate meaningful responses in real-time, aiming for response times within a few seconds.
2) **User-Friendliness**
The program should offer a user-friendly experience through its GUI.
It should provide clear instructions on how to interact with the chatbot and navigate the GUI, making it accessible to users with varying levels of technical expertise as well.

5.2. Design

The program's fundamental design choices are described below.

5.2.1. Creating a sufficient dataset. Since the search for a dataset that meets all the requirements has not been successful, the decision was made to create one. To do so, the following rules need to be considered:

1) **Cryptocurrency Diversity**
The dataset should cover a wide range of cryptocurrency topics, including blockchain technology, various cryptocurrencies (e.g., Bitcoin, Ethereum, e.t.c.), and their global economic impact.
2) **Content Quality**
The questions and answers in the dataset have to be accurate, informative, and relevant. This requires careful research and gathering information from trusted sources.
3) **Variety of User Queries**
To effectively train a chatbot, it needs to be considered diverse user queries. The dataset should include questions on cryptocurrency basics, trading strategies, investment insights, and market trends.
4) **Skill Levels**

The dataset should cater to users with different levels of cryptocurrency knowledge, including beginners, intermediate users, and advanced enthusiasts. It should provide explanations suitable for users at various learning stages.

5.2.2. Developing a model. This part describes essential step in order to achieve main goal of the project.

1) **Choosing packages**
   Choosing right packages is important step in developing model because there are a lot of packages that allow to do so but it needs to find the most suitable ones.
2) **Model Architecture Selection**
   Choosing an appropriate architecture for the chatbot model is a central decision. Considering the model's intended use, a suitable LLM is selected as the base architecture.
3) **Fine-Tuning Strategy**
   The selected pretrained model is prepared for fine-tuning by modifying its architecture, adding task-specific layers, or fine-tuning existing layers. The architecture should be adaptable to understanding cryptocurrency-related questions and generating informative responses.
4) **Hyperparameter Tuning**
   Hyperparameters such as the learning rate, batch size, and dropout rates are systematically adjusted and fine-tuned. Hyperparameter tuning plays a crucial role in optimizing the model's convergence and performance.
5) **Merging the models**
   The fine-tuned model must be merged with the basic model in order to be stored.
6) **Uploading a model**
   It is good practice to upload new model to some service that allow to store custom models. In this way, it is easier to download and use in the future.

By adhering to these data preprocessing and model development steps, the educational cryptocurrency chatbot's design ensures that it can effectively process user queries and provide informative responses on cryptocurrency topics.

5.2.3. Interaction with model using GUI. This subsection is about how to fulfill second functional requirement.

1) **Generating answer**
   Once model is ready, it can be downloaded and tested. The test would be simple conversation with a model through generating an answer to the query from it.
2) **User Interface Design**
   Designing the GUI is important step in the development of the educational cryptocurrency chatbot program. The GUI serves as the user-friendly front-end, enabling users to interact with the chatbot easily.
3) **GUI Components**
   The GUI is comprised of various components designed to enhance user experience and engagement. These components may include input fields for users to enter queries or questions related to cryptocurrency, output panels to display chatbot responses, and interactive controls such as buttons or options for initiating conversations and accessing specific features.
4) **User Experience Enhancement**
   User experience principles will be considered during the GUI design phase. The layout and visual elements will be designed to be pleasing and intuitive, ensuring that users can navigate the interface effortlessly. Factors such as font choice, color schemes and so on will be taken into account to optimize user satisfaction.

By developing an intuitive and visually appealing Graphical User Interface, users can engage with the educational cryptocurrency chatbot effortlessly, enabling them to access valuable information and insights about cryptocurrency topics in an interactive way.

5.2.4. Utilizing RAG technique. This part describes another significant approach in creating a chatbot.

1) **Indexing**
   First of all, it needs to load created dataset.
   Once dataset is loaded, it needs to be splitted and then stored in the vector space, optimizing it for quick and accurate retrieval.
2) **Retrieving**
   Since the dataset now is stored in the vector space, it requires to search necessary information through it. That what retrieving stands for. It finds and provides the given information among all the information.
3) **Generation**
   Once, information is retrieved it can be passed to the model in order to generate an answer to the query.
4) **Adding chat history**
   To get full-fledged chatbot that can communicate in human-like way, it requires to create a chat history to be able to store previous conversation, maintaining continuity and context in ongoing dialogues.

5.3. Production

This section describes details about creating a program. All the code was written in Google Colab and can be seen here:

5.3.1. Creating a sufficient dataset. Dataset uses `.jsonl` format because it was created for the particular pre-trained model which name is LLama2 [6] and it supports only chosen format and `.csv`. Dataset represents a set of queries and responses and it consists of 238 lines. Each line is a dictionary with a single key-value pair, where the key is `text` and the associated `value` is a string containing a query and response. Associated value was also constructed in a understandable way for Llama2. Particularly, it indicates beginning and the end of both queries and responses. This way of creating datasets calls prompt engineering, because, basically, dataset just gives the prompts to the pre-trained model.

Additionally dataset fulfilles set of recommended rules, namely:

- The dataset covers a wide range of cryptocurrency topics that is according to the **Cryptocurrency Diversity** recommended rule
- The questions and answers in the dataset are accurate, informative, and relevant which was done according to **Content Quality** so-called rule
- Dataset also satisfies **Variety of User Queries** condition by including diverse user queries.

To be able to use created dataset in further steps, it was uploaded to the `huggingface.co`

5.3.2. Developing a model.

Used Packages

The following packages were used to implement the program:

1) `Transformers` is library for Python that was created by HuggingFace. It provides tools to easily download and train state-of-the-art pretrained models.
2) `Torch` library is a core part of the PyTorch deep learning framework. It includes data structures designed for handling multi-dimensional tensors and defines mathematical operations performed on these tensors. Furthermore, it offers a range of utilities for efficiently serializing tensors and various data types, along with additional tools for diverse functionalities.
3) `Datasets` is a library for easily accessing and sharing datasets for Audio, Computer Vision, and Natural Language Processing (NLP) tasks. With help of this library it is easy to download the dataset from HuggingFace.
4) `PEFT`, which stands for Parameter-Efficient Fine-Tuning, is a library designed to adapt pre-trained language models (PLMs) for diverse downstream applications without the need to fine-tune the entire set of model parameters. PEFT techniques focus on fine-tuning only a limited number of additional model parameters, leading to substantial reductions in computational and storage expenses. This is particularly advantageous as fine-tuning extensive PLMs can be excessively resource-intensive. Recent advancements in PEFT approaches demonstrate performance levels comparable to those achieved through full fine-tuning.
5) `TRL` is a comprehensive library designed for training transformer language models using Reinforcement Learning. It encompasses tools for every stage of the process, including Supervised Fine-tuning (SFT), Reward Modeling (RM), and Proximal Policy Optimization (PPO). The library seamlessly integrates with the transformers framework.
6) `Bitsandbytes` serves as a streamlined interface for CUDA custom functions, with a specific focus on 8-bit or 4-bit optimizers, matrix multiplication and quantization functions.
7) `Accelerate` is a library that facilitates running identical PyTorch code across various distributed configurations with just a minimal addition of few lines of code. It simplifies, enhances, and makes scalable training and inference straightforward and adaptable.

Model Architecture Selection

Developing a model starts with downloading a base model which is Llama2. The model that was used in this project was trained on 7 billion parameters. There are also Llama 2 models with 13 billion and 70 billion parameters but they are require very powerful computers to be able to work with them. The next step is to fine-tune the model. The following code was used to implement those steps See Listing 2.
Let's get trough the code part by part:

To implement the program these crucial libraries were utilized using the code below:

```
import os
import torch
from transformers import AutoModelForCausalLM,
     AutoTokenizer, BitsAndBytesConfig,
    TrainingArguments, pipeline, logging,
    HfArgumentParser
from peft import LoraConfig, PeftModel
from trl import SFTTrainer
from datasets import load_dataset
```

Next block defines base model, dataset and name of the new model:

```
model_name = "NousResearch/Llama-2-7b-chat-hf"
new_model = "Llama-2-7b-chat-bot"
dataset_name = "siulhin-vlad37/chatbot"
```

`model_name` is the model that was chosen from HuggingFace hub
`dataset_name` is the dataset that was uploaded by author to the HuggingFace hub
`new_model` is the model that will be created while implementing the program

Fine-tuning strategy

This section loads dataset and adjusts Bits and Bytes configuration. `load_dataset` is a function from transformers library that allows to load dataset from HuggingFace. Next lines of code are setting parameters for 4-bit quantization:

```
dataset = load_dataset(dataset_name, split="
    train")

compute_dtype = getattr(torch, "float16")

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="float16",
    bnb_4bit_use_double_quant=False,
)
```

- NF4 (Normal Float 4) data type is a 4bit datatype adapted for weights that have been initialized using a normal distribution.

- This parameter, `bnb_4bit_compute_dtype`, specifies the data type to be used for computations during the 4-bit quantization process. In this case, it is set to `float16`, indicating the use of 16-bit floating-point numbers for intermediate computations.
- This parameter, `bnb_4bit_use_double_quant`, is a boolean flag that controls whether double quantization is used. It is set to false because nested quantization is used instead.

Loads base model and sets it up:

```
model = AutoModelForCausalLM.from_pretrained(
    model_name, quantization_config=bnb_config
    , device_map={"": 0})
model.config.use_cache = False
model.config.pretraining_tp = 1
```

`model` loads Llama2 model from HuggingFace, specifying the name, quantification configuration that was adjusted in the previous step
`device_map` transfers the entire model to GPU 0. `use_cache` is False because when fine-tuning the model, it is important to use the updated model parameters. Using cached values is meaningless in fine-tuning.
`pretraining_tp` is set to 1 to disable a more precise but slower computation for the linear layers. This adjustment is intended to closely align with the original logits for enhanced accuracy.

The following code loads tokenizer and setting up additional settings related to it:

```
tokenizer = AutoTokenizer.from_pretrained(
    model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
```

`tokenizer` loads tokenizer from HuggingFace, specifying model that is used in this project, `trust_remote_code` allows to use a model with custom code.
`pad_token` is a unique token utilized to standardize the size of token arrays for batching purposes. In this case is set to `eos` that is a special token representing the end of a sentence.
`padding_side` refers to the side on which padding should be applied to the model, in this case to the right side.

Hyperparameter Tuning

This block of code adjusts `training_arguments` parameters.

```
training_arguments = TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=1,
    max_grad_norm=0.3,
    optim="paged_adamw_32bit",
    lr_scheduler_type="cosine",
    save_steps=0,
    logging_steps=25,
    max_steps=-1,
    weight_decay=0.001,
    learning_rate=2e-4,
    fp16=False,
    bf16=False,
    warmup_ratio=0.03,
    group_by_length=True,
    report_to="tensorboard"
)
```

- `output_dir` sets the directory where the model's predictions and checkpoints will be saved. In this case, they will be saved in a folder named "results" in the current directory.
- `num_train_epochs` specifies the number of training epochs. An epoch is one complete pass through the entire training dataset.
- `per_device_train_batch_size` and `per_device_eval_batch_size` set the batch size for training and evaluation respectively on each GPU. Batch size refers to the number of samples that will be propagated through the network at one time.
- `gradient_accumulation_steps`: sets how many steps to accumulate gradients before updating model parameters.
- `max_grad_norm` sets a limit on the maximum value of gradients. It helps in preventing explosion of gradients which can result in NaN values during training.
- `optim="paged_adamw_32bit"` the optimizer used here is a variant of AdamW with paged memory management scheme and uses 32 bit precision.
- `lr_scheduler_type` sets the learning rate schedule to be a cosine schedule.
- `save_steps`, `logging_steps` these lines control how often model checkpoints are saved and logs are printed respectively. In this case, model checkpoints are not saved during training but logs are printed every 25 steps.
- `max_steps` sets the total number of training steps. If set to -1, it will be determined by the number of epochs and the size of the training dataset.
- `weight_decay` regularization technique which discourages large weights in the model via a penalty added to loss function.
- `learning_rate` sets initial learning rate for AdamW optimizer which is used for weight updates.
- `fp16` and `bf16` disable mixed-precision training using 16-bit floating point numbers or brain floating point 16. This can speed up training and reduce memory usage.
- `warmup_ratio` sets the ratio of total steps for linear warmup. It helps model to adapt with learning process gradually.
- `group_by_length` when true, sequences are grouped into batches with similar lengths. It saves memory and speeds up training.
- `report_to` is set to `tensorboard` which serves as

a tool for monitoring and visualizing metrics, in addition to providing visualizations for models.

These lines of code tuning parameters for a LoRa-based system implemented through QLoRa approach:

```
peft_config = LoraConfig(
    lora_alpha=16,
    lora_dropout=0.1,
    r=64,
    bias="none",
    task_type="CAUSAL_LM",
)
```

- `lora_alpha` is the scaling factor for the weight matrices.
- `lora_dropout` parameter is set to 0.1 which indicates the dropout rate in the model to 0.1 (or 10%). Dropout is a technique used in neural networks to prevent overfitting by randomly ignoring some neurons during training.
- The `r` stands for rate that is set to 64.

This section adjusts training parameters. `trainer` allows to fine-tune the model on chosen dataset. Supervised fine-tuning parameters as follows:

```
trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    peft_config=peft_config,
    dataset_text_field="text",
    max_seq_length=None,
    tokenizer=tokenizer,
    args=training_arguments,
    packing=False,
)
```

- `model` is a Llama 2 model from HuggingFace.
- `train_dataset` is set to created dataset that was downloaded from HuggingFace.
- `peft_config` is set to defined peft_config
- `dataset_text_field` is set to ”text” that is name of the field in the dataset that contains the text.
- `max_seq_length` is the maximum sequence length of the inputs. It is set to None because there is no need to limit the sequence.
- `tokenizer` is set to earlier adjusted tokenizer.
- `args` is set to defined training arguments.
- `packing` is False since `ConstantLengthDataset` is not utilized in this case.

Beginning of the training defined here:

```
trainer.train()
trainer.model.save_pretrained(new_model)
```

`train` starts training.
`save_pretrained` saves trained model.

For convenience and credibility the dicision was made to visualize the training process. It was done as follows:

```
%load_ext tensorboard
%tensorboard --logdir results/runs
```

Visualization of training. See Images

Emptying VRAM to be able to finish further steps:

```
del model
del pipe
del trainer
import gc
gc.collect()
gc.collect()
```

Clearing VRAM.

Merging new model with base model

To be able to store fine-tuned model it needs to merge it with base model, to do so this code was used:

```
base_model = AutoModelForCausalLM.
    from_pretrained(model_name,
    low_cpu_mem_usage=True, return_dict=True,
    torch_dtype=torch.float16, device_map={"":
     0})
model = PeftModel.from_pretrained(base_model,
    new_model)
model = model.merge_and_unload()

tokenizer = AutoTokenizer.from_pretrained(
    model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
```

First of all the base model is loaded again.See Then, new model and the base one are merged and unloaded using this line of code: `merge_and_unload`. The rest of the code is the same as was described here.

Uploading model

For the sake of convenience new model was uploaded to HuggingFace using following code:

```
import locale
locale.getpreferredencoding = lambda: "UTF-8"

!huggingface-cli login
model.push_to_hub("siulhin-vlad37/Llama-2-7b-
    chat-final", check_pr=True)
tokenizer.push_to_hub("siulhin-vlad37/Llama
    -2-7b-chat-final",check_pr=True)
```

`login` is used to login to the HuggingFace account and enter the generated token to be able to upload the model there.
`push_to_hub` pushes new model and tokenizer to the HuggingFace hub.

5.3.3. Interaction with model using GUI. Once the model is fine-tuned, we can interact with it. See Listing.
Below is example of how to do so:

Importing libraries including the ones for creating Graphical User Interface.

```
import transformers
import torch
from transformers import AutoTokenizer
from ipywidgets import widgets, Layout
from IPython.display import display
```

Defines `model, prompt_text, output_text`

```
model = "siulhin-vlad37/Llama-2-7b-chat-bot"

prompt_text = widgets.Text(
    placeholder="Enter prompt",
    description="Prompt:",
    layout=Layout(width="50%"),
)

output_text = widgets.Textarea(
    value="",
    placeholder="Generated Text",
    description="Generated Text:",
    layout=Layout(width="70%", height="100px"),
)
```

- `model` defines fine-tuned model.
- `prompt_text` equal to `widgets.Text` which creates a text input widget using the `Text` class from the widgets module. It allows users to input text interactively.
  1) `placeholder` sets a placeholder text that will be displayed in the text input box when it is empty. In this case, it suggests to the user to enter a prompt.
  2) `description` sets a description for the text input box.
  3) `layout` sets the layout properties for the text input widget. It specifies that the width of the widget should be 50% of the available space.
- `output_text` equals to `widgets.Textarea` that creates a textarea widget using the `Textarea` class from the widgets module. The Textarea widget allows for multiline text input or output.
  1) `value` initializes the initial value of the textarea. In this case, it starts with an empty string.
  2) `placeholder` similar to the text input widget, this sets a placeholder text that will be displayed when the textarea is empty.
  3) `description` sets a description for the textarea. In this case, it labels the textarea with `Generated Text`.
  4) `layout` sets the layout properties for the textarea widget. It specifies that the width is 70% of the available space, and the height is 100 pixels.

The following code generates button, creates function that generates text and make on click event:

```
generate_button = widgets.Button(description="
    Generate")

def generate_text(b):
    prompt = prompt_text.value

    tokenizer = AutoTokenizer.from_pretrained(
        model)
    pipeline = transformers.pipeline(
        "text-generation",
        model=model,
        torch_dtype=torch.float16,
        device_map="auto",
    )

    sequences = pipeline(
        f'[INST] {prompt} [/INST]',
        do_sample=True,
        top_k=10,
        num_return_sequences=1,
        eos_token_id=tokenizer.eos_token_id,
        max_length=500,
    )

    generated_text = sequences[0]['
        generated_text']
    generated_text = generated_text.replace(f'[
        INST] {prompt} [/INST]', '').replace('[
        INST]', '').replace('[/INST]', '')
    output_text.value = generated_text.strip()

generate_button.on_click(generate_text)
```

- `generate_button`: this line creates a button widget using the `Button` class from the widgets module.
- `generate_text` defines a function that takes an argument b which is the button that triggered when the click event happens.
- `prompt`: extracts the text entered into the `prompt_text` widget, which is the input for the text prompt.
- `tokenizer` loads tokenizer for fine-tuned model as before.
- `pipeline` creates a text generation pipeline using the Hugging Face transformers library. It specifies parameters such as the model to be used, the data type for torch tensors, and the device to use (GPU or CPU). Pipelines provide a convenient and straightforward method for leveraging models in the inference process.
- `sequences` generates text sequences using the specified prompt and the configured text generation pipeline. `do_sample` activates decoding strategies like multinomial sampling, beam-search multinomial sampling, Top-K sampling, and Top-p sampling. Each of these strategies involves selecting the next token based on the probability distribution across the entire vocabulary, with specific adjustments tailored to the chosen strategy. `top_k` represents the number of highest probability vocabulary tokens to keep for top-k-filtering. `num_return_sequences` is the number of independently computed returned sequences for each element in the batch. `eos_token_id` the token ID to enforce as the final generated token when the `max_length` is reached. `max_length` restricts the total length of the returned sequence, limiting it at a maximum length.
- `generated_text` this line retrieves the generated text

from the first sequence in the sequences list. Next line removes a specific pattern from the generated text. It replaces the input prompt with an empty string. This is done to clean up the output and remove any artifacts related to the input prompt.

- Finally, the cleaned and processed generated text is assigned to the `value` attribute of the `output_text` widget. The `.strip()` method is used to remove leading and trailing whitespaces from the generated text before assigning it to the widget. This ensures a cleaner presentation of the generated text in the user interface. `generate_button.on_click` sets up an event handler for the `on_click` event of the `generate_button` widget.

This code displays field for prompt input, button and field for output:

```
display(prompt_text)
display(generate_button)
display(output_text)
```

Chatbot view can be seen here.

5.3.4. Utilizing RAG technique. The RAG technique is based on the GPT-4. This part of technical deliverable describes another approach of developing a chatbot.

Packages and Key Configuration

`OpenAI` library is used to get access to the GPT-4 model. `langchain` library is used for the rest.

This code sets environment variable which is an API key for the GPT-4 model.

```
os.environ['OPENAI_API_KEY'] = 'secret-api-key'

client = OpenAI(
  api_key=os.environ['OPENAI_API_KEY'],
)
```

Indexing

The following code loads dataset from HuggingFace and insert it into vector space:

```
dataset_name = "siulhin-vlad37/final"
page_content_column = "text"

loader = HuggingFaceDatasetLoader(dataset_name, page_content_column)

data = loader.load()

vectorstore = Chroma.from_documents(documents=data, embedding=OpenAIEmbeddings())
```

Once dataset is loaded, the `data` variable holds the dataset content, which is then passed to a `Chroma` vector store. The `Chroma.from_documents()` method is used to create vector embeddings of the documents using the `OpenAIEmbeddings()` model. These embeddings can later be used to retrieve information from the dataset based on similarity to input queries.

Retrieving

This code sets up a document retriever that searches for the most relevant documents based on similarity:

```
retriever = vectorstore.as_retriever(
    search_type="similarity", search_kwargs={"
    k": 6})

retrieved_docs = retriever.
    get_relevant_documents(
   "What are the risks of cryptocurrency?"
)

print(retrieved_docs[0].page_content)
```

`retriever` creates a retriever from the previously built `vectorstore`. The retriever uses similarity search to find documents related to a query.
The `search_kwargs="k": 6` argument specifies that the retriever should return the 6 most similar documents to the given query.
Variable `retrieved_docs` stores the result of calling `retriever.get_relevant_documents()`, which retrieves documents relevant to the question.

Generating

This code snippet is configuring a pipeline for a chatbot that uses the GPT-4 model for generating responses:

```
llm = ChatOpenAI(model_name="gpt-4",
    temperature=0)

prompt = hub.pull("rlm/rag-prompt")

print(
   prompt.invoke(
      {"context": "filler context", "question"
          : "filler question"}
   ).to_string()
)

def format_docs(data):
   return "\n\n".join(doc.page_content for doc
        in data)

rag_chain = (
   {"context": retriever | format_docs, "
       question": RunnablePassthrough()}
   | prompt
   | llm
   | StrOutputParser()
)

for chunk in rag_chain.stream("What is Crypto
    Regulation?"):
   print(chunk, end="", flush=True)
```

`llm` initializes a chat model using GPT-4 with a specified temperature of 0, which stands for the randomness of the

response, 0 is deterministic.
`prompt` retrieves a predefined RAG prompt template from the `LangChain` library's hub.
`print` executes the prompt with a filler context and question and prints the generated string to the console.
`format_docs` defines a function to format the documents by joining their content with new lines between them.
`rag_chain` sets up a pipeline where the retriever gets documents based on a query, formats them, feeds them into the prompt, processes the result through GPT-4, and finally parses the string output.
Line number 56 is a loop that streams the response from the `rag_chain` and prints each part of the response as it is generated, without waiting for the entire response to be constructed.

Adding chat history

This code snippet is for a chatbot that can process user questions by referring to a chat history to create contextually relevant questions and answers. Code can be seen here.

`condense_q_system_prompt` defines a prompt asking the system to reformulate a question based on the chat history, making the question stand alone without needing the context of the chat history.
`condense_q_prompt` creates a template for the chat prompts using the defined system prompt and placeholders for the chat history and user question.
`condense_q_chain`: this chain reformulates the question, if necessary, using the GPT-4 model and parses the result into a string.
`qa_system_prompt` sets up a prompt for answering questions using provided context, adhering to brevity and clarity.
`condense_question` is a function that decides whether to reformulate the question based on the chat history or to use the question as it is.
`rag_chain` is the main pipeline for the chatbot, which assigns context to the retrieved and formatted documents, prompts for question answering, and then uses llm to generate a response.
The line number 89 is the loop that continuously takes user input, processes it through the `rag_chain`, and outputs the chatbot's response, appending each interaction to the chat history for context. It stops if the user types "exit," "quit," or "stop."

5.4. Assessment

The program is a Python-based application with a Graphical User Interface (GUI) designed to serve as a cryptocurrency information chatbot. It meets the following *functional requirements*:

1) **Providing User with Meaningful Responses**:
   The chatbot effectively delivers accurate and informative responses to user queries regarding cryptocurrency, using DL, NLP, LLM and RAG techniques for effective understanding and interpretation of user input.
2) **Graphical User Interface**
   The program successfully incorporates a user-friendly GUI, providing an intuitive design and layout for easy interaction with the cryptocurrency chatbot. The GUI ensures a positive user experience when accessing cryptocurrency information.

The program also satisfies the *non-functional requirements*:

1) **Responsiveness**:
   The chatbot responds promptly to user queries, minimizing lag in providing answers. It processes user inputs in real-time, aiming for response times within a few seconds to enhance user interaction.
2) **User-Friendliness**:
   The program offers a user-friendly experience through its GUI, providing clear instructions on how to interact with the chatbot and navigate the interface. This ensures accessibility for users with varying levels of technical expertise.

In conclusion, the program meets the specified functional and non-functional requirements, providing an effective and user-friendly solution for obtaining cryptocurrency information through a GUI.

Acknowledgment

The author would like to thank Salima LAMSIYAH, BiCS management and education team for the amazing work done.

6. Conclusion

AI-based chatbots became popular for the last time. There are many of them and each specialises on something narrowly focused. The aim of this project is to learn how to fine-tune LLM and to practice it. It is going to help to train a chatbot to answer meaningfully to the answers that he could not answer before.

In conclusion, this project successfully answered to the scientific question and also programs were satisfactorily implemented.

Through the utilization of Deep Learning, NLP, LLM and RAG techniques, the chatbot adeptly responds to user queries, showcasing its effectiveness in providing valuable interactions.

7. Plagiarism statement

I declare that I am aware of the following facts:

- As a student at the University of Luxembourg I must respect the rules of intellectual honesty, in particular not to resort to plagiarism, fraud or any other method that is illegal or contrary to scientific integrity.
- My report will be checked for plagiarism and if the plagiarism check is positive, an internal procedure will

be started by my tutor. I am advised to request a pre-check by my tutor to avoid any issue.

- As declared in the assessment procedure of the University of Luxembourg, plagiarism is committed whenever the source of information used in an assignment, research report, paper or otherwise published/circulated piece of work is not properly acknowledged. In other words, plagiarism is the passing off as one's own the words, ideas or work of another person, without attribution to the author. The omission of such proper acknowledgement amounts to claiming authorship for the work of another person. Plagiarism is committed regardless of the language of the original work used. Plagiarism can be deliberate or accidental. Instances of plagiarism include, but are not limited to:
  1) Not putting quotation marks around a quote from another person's work
  2) Pretending to paraphrase while in fact quoting
  3) Citing incorrectly or incompletely
  4) Failing to cite the source of a quoted or paraphrased work
  5) Copying/reproducing sections of another person's work without acknowledging the source
  6) Paraphrasing another person's work without acknowledging the source
  7) Having another person write/author a work for oneself and submitting/publishing it (with permission, with or without compensation) in one's own name ('ghost-writing')
  8) Using another person's unpublished work without attribution and permission ('stealing')
  9) Presenting a piece of work as one's own that contains a high proportion of quoted/copied or paraphrased text (images, graphs, etc.), even if adequately referenced

  Auto- or self-plagiarism, that is the reproduction of (portions of a) text previously written by the author without citing that text, i.e. passing previously authored text as new, may be regarded as fraud if deemed sufficiently severe.

References

[1] F. J. D. Nitin Indurkhya, *Handbook of Natural Language Processing*. CRC Press LLC, 1999, Link.

[2] A. Lewis, *The basics of bitcoins and blockchains : an introduction to cryptocurrencies and the technology that powers them*. Coral Gables, FL: Mango, 2021, Link.

[3] R. Khan and A. Das, *Build better chatbots : : a complete guide to getting started with chatbots /*. Berkeley, CA: Apress, 2018, Link.

[4] "What is a large language model (llm)?" *elastic*, 2023, Link.

[5] "Retrieval-augmented generation (rag)," *LangChain*, 2023, Link.

[6] Meta, "Introducing llama 2," 2023, Link.

8. Appendix

Additional Images

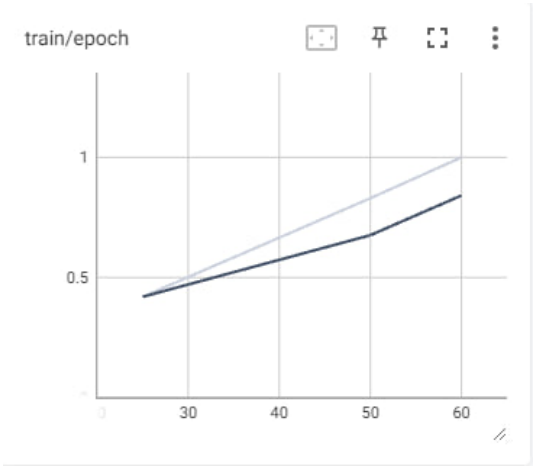


Figure 1

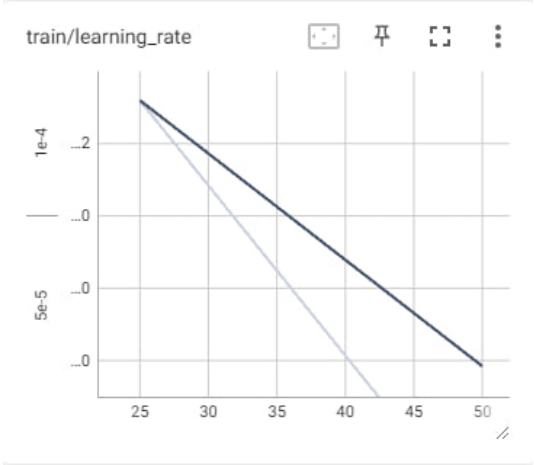


Figure 2

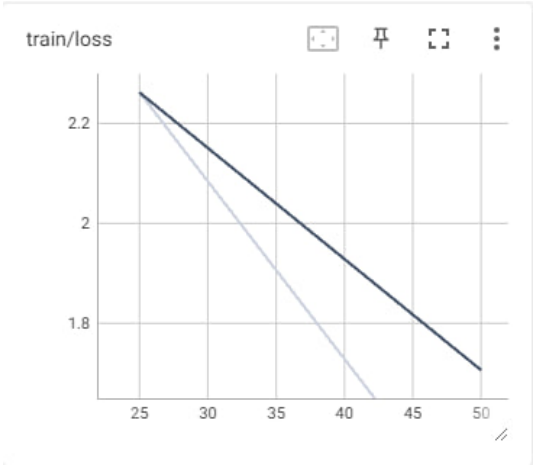


Figure 3

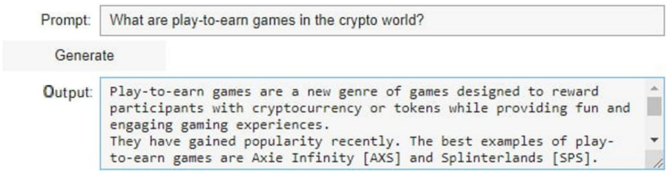


Figure 4

8.1. Source code

```
!pip install accelerate peft bitsandbytes
    transformers trl
```

Listing 1: Installing libraries

```
import os
import torch
from transformers import AutoModelForCausalLM,
     AutoTokenizer, BitsAndBytesConfig,
    HfArgumentParser, TrainingArguments,
    pipeline, logging
from datasets import load_dataset
from trl import SFTTrainer
from peft import LoraConfig, PeftModel
```

Listing 2: Importing libraries

```
model_name = "NousResearch/Llama-2-7b-chat-hf"
dataset_name = "siulhin-vlad37/chatbot"
new_model = "Llama-2-7b-chat-final"

dataset = load_dataset(dataset_name, split="
    train")

compute_dtype = getattr(torch, "float16")

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="float16",
    bnb_4bit_use_double_quant=False,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map={"": 0}
)
model.config.use_cache = False
model.config.pretraining_tp = 1

tokenizer = AutoTokenizer.from_pretrained(
     model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

peft_config = LoraConfig(
    lora_alpha=16,
    lora_dropout=0.1,
    r=64,
    bias="none",
    task_type="CAUSAL_LM",
)

training_arguments = TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=1,
    optim="paged_adamw_32bit",
    save_steps=0,
    logging_steps=25,
    learning_rate=2e-4,
```

```
    weight_decay=0.001,
    fp16=False,
    bf16=False,
    max_grad_norm=0.3,
    max_steps=-1,
    warmup_ratio=0.03,
    group_by_length=True,
    lr_scheduler_type="cosine",
    report_to="tensorboard"
)

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    peft_config=peft_config,
    dataset_text_field="text",
    max_seq_length=None,
    tokenizer=tokenizer,
    args=training_arguments,
    packing=False,
)

trainer.train()
trainer.model.save_pretrained(new_model)
```

Listing 3: Fine-tuning the model

```
%load_ext tensorboard
%tensorboard --logdir results/runs
```

Listing 4: Visualization of training

```
del model
del pipe
del trainer
import gc
gc.collect()
gc.collect()
```

Listing 5: Clearing VRAM

```
base_model = AutoModelForCausalLM.
    from_pretrained(
    model_name,
    low_cpu_mem_usage=True,
    return_dict=True,
    torch_dtype=torch.float16,
    device_map={"": 0},
)
model = PeftModel.from_pretrained(base_model,
    new_model)
model = model.merge_and_unload()

tokenizer = AutoTokenizer.from_pretrained(
    model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
```

Listing 6: Merging two models

```
import locale
locale.getpreferredencoding = lambda: "UTF-8"

!huggingface-cli login
model.push_to_hub("siulhin-vlad37/Llama-2-7b-
    chat-final", check_pr=True)
tokenizer.push_to_hub("siulhin-vlad37/Llama
    -2-7b-chat-final",check_pr=True)
```

Listing 7: Pushing model and tokenizer to HuggingFace

```
from ipywidgets import widgets, Layout
from IPython.display import display
from transformers import AutoTokenizer
import transformers
import torch

model = "siulhin-vlad37/Llama-2-7b-chat-final"

prompt_text = widgets.Text(
    placeholder="Enter prompt",
    description="Prompt:",
    layout=Layout(width="50%"),
)

output_text = widgets.Textarea(
    value="",
    placeholder="Generated Text",
    description="Generated Text:",
    layout=Layout(width="70%", height="100px"),
)

generate_button = widgets.Button(description="
    Generate")

def generate_text(b):
    prompt = prompt_text.value

    tokenizer = AutoTokenizer.from_pretrained(
        model)
    pipeline = transformers.pipeline(
        "text-generation",
        model=model,
        torch_dtype=torch.float16,
        device_map="auto",
    )

    sequences = pipeline(
        f'[INST] {prompt} [/INST]',
        do_sample=True,
        top_k=10,
        num_return_sequences=1,
        eos_token_id=tokenizer.eos_token_id,
        max_length=500,
    )

    generated_text = sequences[0]['
        generated_text']
    generated_text = generated_text.replace(f'[
        INST] {prompt} [/INST]', '').replace('[
        INST]', '').replace('[/INST]', '')
    output_text.value = generated_text.strip()

generate_button.on_click(generate_text)

display(prompt_text)
display(generate_button)
display(output_text)
```

Listing 8: Interaction with model

```
condensed_q_system_prompt = """Given a chat
    history and the most recent user query \
```

```
which which may relate to the chat history, formulate a standalone question \
without needing background from the chat history. Do NOT provide an answer, \
just reformulate it if necessary and otherwise return it as is."""
condense_q_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", condense_q_system_prompt),
        MessagesPlaceholder(variable_name="chat_history"),
        ("human", "{question}"),
    ]
)
condense_q_chain = condense_q_prompt | llm | StrOutputParser()

condense_q_chain.invoke(
    {
        "chat_history": [
            HumanMessage(content="What is ICO?"),
            AIMessage(content="Initial Coin Offering"),
        ],
        "question": "What is meant by Initial",
    }
)

condense_q_chain.invoke(
    {
        "chat_history": [
            HumanMessage(content="What is ICO?"),
            AIMessage(content="Initial Coin Offering"),
        ],
        "question": "How do STO work",
    }
)

qa_system_prompt = """As an assistant tasked with responding to queries, \
utilize the given context to formulate an answer. \
If the answer is unknown, state your lack of knowledge. \
Provide a brief response, limiting it to no more than three sentences.\

{context}"""
qa_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", qa_system_prompt),
        MessagesPlaceholder(variable_name="chat_history"),
        ("human", "{question}"),
    ]
)

def condense_question(input: dict):
    if input.get("chat_history"):
        return condense_q_chain
    else:
        return input["question"]

rag_chain = (
    RunnablePassthrough.assign(context=
        condense_question | retriever | format_docs)
    | qa_prompt
    | llm
)

chat_history = []
while True:
    user_question = input("\nUser: ")
    if user_question.lower() in ["exit", "quit", "stop"]:
        print("\nExiting chatbot.")
        break

    ai_response = rag_chain.invoke({"question": user_question, "chat_history": chat_history})
    print("\nChatbot:", ai_response.content)

    chat_history.extend([HumanMessage(content=user_question), AIMessage(content=ai_response.content)])
```

Listing 9: Adding chat history using RAG